クウハクスコープ 技術仕様書

transit-accessibility ／ 公共交通オープンデータチャレンジ2026 応募作品

2026-10-30 現在（09-28版を改訂） ／ GitHub: K-hskw/transit-accessibility（`main`） ／ 公開アプリ: https://transit-accessibility-aik9qbme9ej2ue8wamnpvf.streamlit.app ／ 対象: 北海道室蘭市・道南バス GTFS-JP（2026年10月1日改正）

> **この文書の目的**：本プロジェクトの文脈を持たない読み手（生成AIを含む）が、コードを見ずに設計判断の当否を議論し、変更提案ができる状態になること。数値はすべて実測値で、`python tools/key_numbers.py` で再計算できる。**2026年5月作成の引き継ぎ文書と、09-28版の本書の数値には、後述する誤りが含まれる**ため、矛盾する場合は本版を優先すること。特に09-28版の時間帯の値は毎時00分の1点で測っており、時刻の取り方に左右されていた（4.4）。

1. 何をするアプリか

GTFSの時刻表から「いつ・どこから・どこへ行けるか」を計算し、**生活拠点に到達できない人口**を地図に可視化する。さらに増便・ダイヤ変更がその人口を何人減らすかを比較する。想定利用者は自治体の交通政策担当者。

1.1 中心概念：時間空白

既存の交通空白の把握は「バス停から半径1km圏外」のような**距離**基準が主流。本作は**時刻表由来の到達可能性**で定義する。同じ地点でも時間帯・曜日によって到達可能性が変わり、距離基準ではこれが見えない。

区分	定義	施策で変わるか
静的空白	徒歩圏（既定500m）にバス停が1つも無いメッシュ	変わらない（停留所新設が必要）
時間空白	バス停はあるが、到着時刻の制限時間前以降に出発して拠点に間に合わないメッシュ	**変わる。本作の測定対象**

2. データ

データ	内容	規模
GTFS-JP（道南バス）	stops / stop_times / trips / calendar / routes / shapes。2026年10月1日改正、有効期間 2026-10-01〜12-25（北海道オープンデータから取得、ODPT基本ライセンス）	停留所 2,182
bus_edges.csv（生成物）	便ごとの停留所間区間。全ダイヤ分を保持	113,730 行
walk_edges.csv（生成物）	停留所間300m以内の徒歩エッジ（双方向）	5,440 行
100mメッシュ人口	簡易100mメッシュ人口・室蘭市（Meshcode, PopT, Pop65over）。令和2年国勢調査の250mメッシュ人口を建物面積で100mメッシュに按分した推計値（作成: 西澤明、CC BY）。1メッシュの値は実人数そのものではない。粒度はメッシュコードの桁数から自動判別（8桁=1km／9桁=500m／10桁=100m）	3,079 メッシュ
facilities.csv	国土数値情報の医療機関・学校（診療所54／歯科40／病院7／小中高・幼稚園・大学等41）。以前の手動収集版から、出典を明示できるデータに差し替えた	142 件

室蘭市 総人口 83,050人 ／ 65歳以上 30,617人（高齢化率 36.9%）。route_id は 106（平日に走る105路線のうち、高速バス等を除いた100が路線選択の対象）。

2.1 ダイヤ種別

`bus_edges.csv` は**全ダイヤ分**を持ち、平日／土曜／日祝の切替は実行時に `trip_id` で絞る。種別ごとにネットワークを作り直さない。

種別	便数	エッジ数	判定

平日	808	32,536	calendar.txt の monday〜friday のいずれかが1
土曜	672	27,143	saturday が1
日祝	667	26,973	sunday が1

土曜と日祝は trip_id が異なるが、室蘭市内の時刻表は同じ。違いは土曜だけ走る5便（伊達市内・留寿都方面）のみ。09-28版は「同一」と書いたが、正確には上記。`calendar_dates.txt`（特定日の運休・追加、お盆などの臨時ダイヤ）は未対応で、祝日ダイヤは「日祝」のservice_idで近似している。

3. モジュール構成

```
service_calendar.py  (53行)  ダイヤ種別の判定。他モジュールに依存しない
        ↑
build_network.py    (133行)  GTFS → bus_edges.csv / walk_edges.csv（1回だけ実行）
transit_engine.py   (845行)  ★探索エンジン。アルゴリズム本体は47行
population.py       (250行)  メッシュ人口・施設。距離計算はnumpyベクトル化
        ↑
blank_area.py       (363行)  空白人口の算出。5分刻みの1日分の診断（DayDiagnosis）
prescription.py     (173行)  処方の評価（engine/analyzer を引数で受け取る）
policy_report.py    (448行)  比較結果 → 政策文書（Markdown）。生成AIは任意
session_data.py      (83行)  訪問者ごとのアップロード処理
        ↑
app.py             (1550行)  Streamlit UI。上記すべてを使う
tools/key_numbers.py         README・資料の数値を再計算する
```

依存は一方向。`prescription.py` は engine を import せず引数で受け取るため単体で差し替え可能。

4. アルゴリズム

4.1 時刻依存ダイクストラ法（`_dijkstra`）

通常の最短経路は辺の重みが固定だが、バスは**何時にその停留所へ着いたかで次の一手のコストが変わる**。8:15着なら8:20発に乗れる（待ち5分）が、8:25着なら9:00発まで待つ（待ち35分）。

実装の要点

- グラフの辺は抽象的な「区間」ではなく**便ごとの具体的な発車**。`bus_graph[stop] = [(dep_sec, arr_sec, to_stop, trip_id), ...]` を発車時刻順にソートして保持
- 緩和条件は `dep_sec >= current_time` のみ。**待ち時間は式に現れず、この制約から自動的に生じる**
- 最小化する量は距離ではなく**到着時刻**。優先度付きキューも時刻順
- 状態は停留所ではなく**（停留所, 直前の移動手段）**の組。`last_mode != "walk"` により**徒歩の連続を禁止**する。これが無いと300m徒歩エッジを連鎖して徒歩だけで市内を横断できてしまう
- 深夜便対応として時刻は24時超も秒で表現（例 25:30 = 91800秒）

> **追い越しに強い**：便を1本ずつ列挙しているため、急行が各停を追い越す場合も両方が候補に入り早く着く方が選ばれる。**FIFO性（追い越しなし）の仮定を必要としない。**
> **注意**：引き継ぎ文書の「急行便は自動除外しFIFO性を保持」は誤り。高速バスは経路探索に**使われている。**
> `get_muroran_routes(exclude_highway=True)` が除外するのはUIの路線選択リストのみで、除外理由は「市内交通と性質が異なり廃止シミュレーションの対象として不適切」というモデル上の判断。アルゴリズムの正当性のためではない。

4.2 逆方向到達圏（`calc_reverse_isochrone`）

向きを逆にし、「**指定時刻までに拠点へ着くための最も遅い出発時刻**」**を最大化**する。バスエッジを `to_stop → (arr_sec, dep_sec, from_stop)` で辿り、`arr_sec <= current_time` かつ `dep_sec >= earliest_time` を満たす便を緩和する。順方向と同じく徒歩の連続を禁止する。

なぜ重要か：空白人口は「全メッシュから拠点へ行けるか」なので、順方向だと3,079メッシュ分の探索が要る。逆方向なら**拠点から1回**で全メッシュ分が出る。3,079メッシュ×216通りの到着時刻（5分刻み）の診断が約1秒で終わるのはこのため（1時刻あたり約4ms）。計算量だけでなく、問いの立て方の変更でもある。

4.3 空白人口の算出（`BlankAreaAnalyzer`）

```
1. 事前計算: メッシュ重心×停留所のうち徒歩圏(500m)に入る組を保持（19,408組）
2. 拠点から逆方向到達圏 → 各停留所の「最も遅い出発時刻」
3. メッシュごとに max(停留所の最遅出発時刻 − そこまでの徒歩時間)
4. それが（到着時刻 − 制限時間）以上なら到達可能、未満なら空白
```

- 徒歩圏に停留所が1つも無いメッシュ＝**静的空白**（対応表に現れないので自然に区別される）
- **メッシュ重心→最寄り停留所の徒歩区間はここで初めて入る**。bus_edges/walk_edges には停留所間の徒歩しか無く、「そもそもバス停まで歩けるか」は評価されていなかった
- 拠点IDが空・未知なら `ValueError`。存在しない停留所名を渡した事故が「空白人口＝総人口」として黙って通るのを防ぐため
- **判定の意味**：到着時刻 T の制限時間前（T−60分）以降に出発し、T までに着けるか。早く着いて拠点で待つ時間も60分に含む（始発で6時に着いて9時まで待つ人を「9時に着ける」と数えると時間空白が見えなくなるため）。そのため T を遅らせても空白が減るとは限らない（休日 7:45着17.5% → 8:00着27.5%）。09-28版と政策文書は「所要時間が60分以内」と説明しており、実装と食い違っていた
- 歩ける距離・時間は直線距離で見る。道のりを直線の1.3倍とすると休日7:00着の空白は46.1%→53.8%、静的空白は3,793→7,613人
- `served_share()`：歩ける範囲にバス停がある人口の割合。人口と時刻表の地域がずれていると0に近づき（室蘭は95%）、アプリは警告を出す。以前は別の都市のGTFSに室蘭の人口が残ったままでも、黙って空白人口100%と表示された

4.4 到着時刻の5分刻みと時間帯の平均（`DayDiagnosis`）

同じ場所でも、到着時刻が数分違うだけで間に合う便があるかが変わる。休日に東室蘭駅東口へ着けない人は、7:00着で46.1%、7:15着で28.2%、7:35着で11.9%。09-28版までは毎時00分の1点だけで測っていたため、**00分がたまたまその時間帯の最悪・最良の時刻だと、時間帯の値を取り違えた**（「休日7時に46.1%」は7時台で最も悪い1分の値だった）。

- 到着時刻を5:00〜22:55の5分刻み（216通り）で判定し、`blank[時刻, メッシュ]` の真偽値行列として持つ
- 時間帯（H時台＝H:00〜H:55着の12通り）の空白人口は平均。メッシュの空白人口は「人口×その時間帯で着けなかった時刻の割合」で、足すと時間帯の平均に一致する（テストで検証）
- 平均とあわせて、時間帯の中の最小・最大と、その到着時刻を必ず示す（休日7時台は平均27.9%、幅11.9〜46.1%）。平均だけ出すと、この揺れ自体＝時間空白が読み手から消える
- 3Dの色（深刻度）は「1日のうち着けない時間」＝着けなかった回数×5分。以前は「00分に空白だった時間帯の数」で、00分の前後で1時間ぶん増減した
- 1分刻みとの差：休日7時台の平均は1分刻みで28.0%、5分刻みで27.9%

5. シナリオ機構（処方の土台）

当初 `simulate_*` は改変エッジを内部で作った直後に前方ダイクストラを回し、**起点1停留所からの到達圏しか返さなかった**。一方、空白人口は逆方向で計算する。そのため「代替路線を引いたら空白人口が何人減るか」が計算できなかった。

各シミュレーションを「改変後エッジを返す関数」と「探索する関数」に分離し、`scenario()` で一時適用できるようにした。

```
before = analyzer.diagnose(dest, t, limit)
with engine.scenario(*engine.edges_after_route_removal(rid)):
    after = analyzer.diagnose(dest, t, limit)   # 改変後のネットワークで診断
# 抜けるとネットワークとグラフキャッシュが復帰（例外時も）
```

改変関数	内容	徒歩エッジ
`edges_after_route_removal`	路線の廃止（単体/複数）	不変
`edges_after_stop_removal`	停留所の削除。路線は通過扱いで維持	変わる
`edges_after_frequency_reduction`	減便（half / interval / all）。停留所パターン（方向・経路）ごとに時刻順で間引く	不変
`edges_after_route_replacement`	路線廃止＋新規路線の追加	不変
`edges_after_frequency_increase`	増便	不変
`edges_after_timetable_shift`	ダイヤシフト（便数不変・費用中立）	不変

`simulate_*` はこれらを呼ぶ薄いラッパとして残っている（既存UIとの互換のため）。テストで両者の結果一致を検証しており、片方だけ直してずれる事故を検出できる。

5.1 増便の設計判断

停留所パターン（=経路と方向の組）ごとに、指定時間帯へ等間隔に便を挿入する。1路線に最大4パターンあるため、方向を無視すると「行きの便を複製して帰りに足す」不整合が起きる。

> **既存便の隙間を埋める方式は採らない。**空白が深刻な時間帯ほど既存便が1本以下で隙間が定義できず、**最も増便したい路線に限って増便できなくなる**ため。運行が皆無の時間帯に新設する場合は `edges_after_route_replacement` を使う。

足す便の時刻は「拠点を通る時刻」で置く（`via_stops`）。拠点を通る系統は、拠点を通る時刻が指定時間帯に等間隔に入るよう始発時刻を逆算し、通らない系統は始発時刻で置く。09-28版は常に始発時刻で置いていたため、既定の「5〜9時に1便」が7:00発になり、7時台に拠点へ着く便にならなかった（候補3路線とも7時の削減が0）。アプリの既定の時間帯も対象の時間帯（例：7時台なら7〜8時）に合わせた。ダイヤシフトの移動元・移動先も同じ基準で判定する。

5.1b 減便の間引き方

便を停留所パターン（方向・経路）ごとに時刻順に並べて間引く。09-28版は路線の便を trip_id 順に1本おきに消しており、**2パターン以上ある53路線中18路線で、片方向の便が全部残るか全部消えていた**（「半分」なのに片方向だけ全廃）。

- half：1本おきに残す（1本目・3本目…）。interval：N本に1本残す（Nは2〜6。以前はNを選ぶ欄が無く常にhalfと同じ結果で、表示は「0本に1本」だった）
- all：パターンごとに k = floor(便数×率 + ϕ) 本を削る。ϕ はパターンごとに固定の0〜1の値（黄金比で散らす）。四捨五入だと便数1〜3本のパターンで切り上げに偏る。実際の削減率は指定の±1ポイント程度（平日30%指定で30.2%、50%指定で49.6%）。09-28版は max(1, int(n×率)) で、平日50%指定で実際は46%だった
- 削る便は、添字+1 を2進数で左右反転した値（van der Corput 列）の小さい順に選ぶ。先頭 k 本はどの k でも時刻に偏らず、k=n//2 ではちょうど2本目・4本目…になる。**率を上げても前に削った便は削られたまま（入れ子）**なので、「減便率を上げれば空白は増えない方向にしか動かない」がデータの偶然ではなく構成として成り立つ

5.2 政策文書の生成（`policy_report.py`）

処方の比較表と診断結果から、自治体向けの文書ドラフト（要旨／現状の診断／比較した施策／推奨／前提条件と限界／データ出典）を組み立てる。**数値と推奨はすべて決定的に算出し、生成AIは使わない。**APIキーが無くても必ず出るので、説明の場でAPIの不調に左右されない。

- 推奨は「全日の削減」が最大の施策。**同点なら運行キロの小さい方**を採る（表の並び順で決めると、削減168で同じでも運行キロ205.7kmと21.7kmのうち高い方を推奨していた）
- 「H時台の空白・削減」はその時間帯の5分刻みの平均（人）、「全日の削減」は5〜22時台の減少を時間で積み上げた値（人・時間）。09-28版は「人・時間帯」という単位で、00分の18点を足していた
- 判定の定義（到着時刻の60分前以降に出発して間に合うか）と、時間帯の中の幅（最小・最大の到着時刻と人数）を本文に書く

- 人口の出典は「簡易100mメッシュ人口（国勢調査を按分した推計値、作成: 西澤明、CC BY）」と書く。09-28版は「100mメッシュ別人口（総務省統計局・e-Stat）」と誤って書いていた（e-Statに100mメッシュの人口は無い）。時刻表の出典はfeed_info.txt から事業者名と有効期間を入れる
- 狙った時刻の削減が0の施策を「0人減る」とは書かず、効果が他の時間帯に出ることを明記する
- 任意で、利用者が自分のAnthropic APIキーを入れると文章だけをClaudeで整える（キーは保存しない）。整形後に**下書きに無い数値が増えていないかを機械的に照合**して表示する。「数値は検証可能、AIは通訳」をこの照合で担保する

5.3 公開環境での訪問者の分離

Streamlit Community Cloud は1つのプロセスを全訪問者で共有する。重いデータ（GTFS・全便エッジ・人口）は共有し、**ダイヤ種別・シナリオ・グラフキャッシュだけを訪問者ごとに分ける**（`TransitEngine.session_copy()`）。状態の変更はすべて属性の再代入で行っているため浅いコピーで分離でき、訪問者1人あたり約20MBで済む。アップロードしたGTFS・人口・施設は訪問者ごとの一時ディレクトリに置き、共有ファイルには書かない。

6. 発見・修正した重大な不具合

いずれも**分析結果を誤らせる**種類のもの。再発させないこと。

6.1 所要0分区間の欠落（最重要）

`build_network.py` が `travel_time > 0` の区間しか採用していなかった。GTFS-JPの時刻は分単位のため、近接停留所間は所要0分になる。

影響	値
除外されていた区間（平日）	9,760 / 115,415（8.46%）
影響を受けた便	1,833 / 2,837
モデル上バスで到達不能だった停留所	**148 → 0**

消えていたのは すずらん団地・みどり団地・ちのみ団地 など、まさに空白人口を測りたい住宅地。**修正は** `>= 0`。時刻表の時刻は絶対値なので0秒エッジを残しても誤差は停留所あたり1分未満で、便の総所要時間には累積しない。時刻逆転（負値）は0件。

6.2 同じ不具合が削除シミュレーション内部にも

`simulate_stop_removal` の通過再接続が `arr_sec > dep_sec`（厳密）で区間を作っていた。**何も削除しなくても到達圏が212 → 91 に壊れる**状態で、削除の影響を大きく過大評価していた。修正後、東室蘭駅東口の削除による到達不能は、室蘭駅前8:00発では 132 → 2箇所（当該停留所のみ）。ただしこれは8:00発だけの値で、8時台を5分刻みで見ると8:35〜8:45発では85停留所が到達不能になる（8時台の平均21）。09-28版の「当該停留所のみ」は8:00発に限った結論だった。

6.3 逆方向到達圏に徒歩の連続禁止が無かった

順方向にある制限が逆方向に無く、300m徒歩エッジを無制限に連鎖できていた。集客圏が18.2%過大（280 → 229停留所）。両者で移動ルールが異なると前後比較が成立しない。

6.4 対応しないモードで共通ボタンを押すとクラッシュ

サイドバーの「シミュレーション実行」を全モードで表示していたが、実行処理は5モードしか扱わず、他のモードで押すと `NameError: result_after` で落ちた。**ブランチ分岐前から存在**していた不具合。ボタンを対応モードのみに限定して修正。

6.5 ゼロ詰めの service_id を数値と取り違える（旭川で発覚）

`calendar.txt` の `"01"` を pandas が整数 1 と推測し、`trips.txt` 側（他の値が混在して文字列のまま）と突合できず、**旭川電気軌道のフィードで全ダイヤが0便**と判定された。室蘭のIDは数値に見えない形式だったため表に出なかった。`dtype={"service_id": str}` で修正し、室蘭の便数（806/674/670）が変わらないことを確認。

6.6 公開環境で訪問者どうしが干渉する

エンジンを全訪問者で共有したまま `set_day_type()` と `scenario()` を呼んでいた。**ある訪問者の処方計算中に別の訪問者が診断を開くと、平日9時の空白人口が 19,867人ではなく 77,690人と表示される**ことを再現した。アップロードも共有ディレクトリに書いていたため、1人がZIPを上げると以後の全訪問者に別の都市が表示されていた。いずれも公開前に 5.3 の方式で修正。

6.7 時間帯の値が毎時00分の1点に依存（2026-10-30 修正）

4.4 を参照。発表に使う予定だった数値（休日7時に46.1%、時間帯格差27.6倍）は、いずれも00分の1点の値だった。27.6倍（5時台11／10時台304）は再現もしなかった（5:00発は10で、当時のコードでも10）。5分刻みの時間帯平均では、休日7時台27.9%、時間帯格差は約7倍。

6.8 減便の間引き方（2026-10-30 修正）

5.1b を参照。片方向だけ全廃になる路線があり、「N本に1本」はNを選べず、一律削減は実際の率が指定とずれていた。README の減便の数値（30%で76箇所など）はこの処理と8:00発の1点で出したもので、出し直した。

6.9 代替路線の帰り方向（2026-10-30 修正）

帰り方向の便に行きと同じ区間時刻を割り当てていたため、帰りは時刻をさかのぼる並びになり、区間ごとに次の便を待つ計算になっていた（5停留所・30分間隔で行き20分・帰り95分）。帰りも始発から時刻を積み上げて作り、行きと帰りの所要時間が一致することをテストで確認。

6.10 施設アクセスの歩行速度（2026-10-30 修正）

「高齢者（分速40m）」を選んでも施設までの徒歩時間は分速67mで計算され、制限時間とも比べず、最寄りの1停留所しか見ていなかった。選んだ速さで歩く時間を含めて制限時間と比べ、500m以内の停留所から最も早く着けるものを使うように修正。

6.11 その他（2026-10-30 修正）

- 改善策の足す便の置き方（5.1）
- モードを切り替えても前のモードの結果（地図と影響分析）が表示され続けた。3D診断・改善策は、条件を変えても再計算するまで警告が出なかった
- アップロードしたGTFSでも、地図の中心が室蘭の座標、路線の線が室蘭の shapes.txt のままだった。人口と時刻表の地域がずれていても警告が出なかった（4.3）
- 有効期間（2026-09-30まで）の切れた時刻表で分析を続けていた。10月1日改正版に更新し、期限切れの時刻表を読み込んだときは画面に警告を出すようにした

7. 引き継ぎ文書（2026年5月）の誤り

古い資料を参照する場合、以下は**使用しないこと**。「引き継ぎ文書」列は 6.1 の欠落ネットワーク上の数値、「09-28版」列は毎時00分の1点（8:00発など）で測った数値。

項目	引き継ぎ文書	09-28版	現在（5分刻みの時間帯平均）
時間帯格差（室蘭駅前・60分）	**13.2倍**	**27.6倍（5時台11 / 10時台304）**	**約7倍（5時台 平均35 / 10時台 平均238）**
路線廃止の影響（室蘭駅前8時台発）	**42路線中5**	**102路線中5（最大72箇所）**	100路線中17（8:00発だけなら5）
一律減便 30%→40%（同）	**14 → 60箇所**	**76 → 109箇所**	平均28 → 38箇所
主要ハブ（東室蘭駅東口）削除（同）	**24箇所が孤立**	**2箇所のみ**	平均21・最大85（8:00発だけなら0）
休日に拠点へ着けない人（7時）	—	**46.1%（7:00着）**	7時台 平均27.9%（11.9〜46.1%）
デマンド交通の効果	**人口 +81%**	—	機能を削除（後述）

バス停通過の時間短縮	**停車時間30秒の削減**	—	誤り。停車時間は全レコードで0秒

8. 現在の主要な計測結果

拠点＝東室蘭駅東口、制限60分、徒歩500m・分速67m。値は到着時刻5分刻みの時間帯平均（カッコ内は割合と、到着時刻による最小〜最大）。

到着の時間帯	平日	土日祝
7時台	18,087（21.8%・14,934〜22,723）	23,183（27.9%・9,867〜38,322）
9時台	18,559（22.3%・11,334〜27,207）	20,196（24.3%・13,825〜27,368）
12時台	24,603（29.6%・17,355〜29,760）	26,311（31.7%・20,326〜33,599）
17時台	24,094（29.0%・15,975〜31,755）	27,118（32.7%・24,051〜31,385）

平日9時台の内訳は 静的空白 3,793人 ＋ 時間空白 14,766人。**時間帯の中の幅が平日・土日祝の差より大きい**ことに注意（休日7時台の幅は9,867〜38,322人）。00分の1点で比べると、この幅のどこを取ったかで大小関係が入れ替わる。**平日と土日は互いに素な時刻表なので、どちらの空白が大きいかは時間帯で入れ替わりうる。**大小関係を決め打ちしてはいけない。

2026年10月1日改正で東室蘭駅東口への到達に関わる便は変わらず、上表の値は4〜9月ダイヤと同じだった。変わったのは伊達・洞爺・日高方面と空港線、室蘭市内の工大循環線・中央町工大循環線の一部の便。

8.1 処方の比較（休日7時台・アプリの既定設定）

候補上位3路線、増便は拠点を7〜8時に通るよう1便/パターン、シフトは10〜15時から7〜8時へ2便/パターン。基準は7時台平均23,183人。

施策	7時台の削減	全日の削減（人・時間）	追加便	運行キロ増
増便 候補3路線すべて	1,328	**4,650**	12	247.6
増便 絵鞆循環線	1,086	**3,727**	9	205.7
増便 10絵鞆・工大線	1,171	**1,255**	1	20.3
シフト 絵鞆循環線	4,932	**−3,694**	0	0.0

09-28版の同じ表（00分の1点で評価）は、「増便 10絵鞆・工大線」が7時の削減0・全日8,847で、2便にすると全日0になる逆転が起きていた。足した便が00分の直前に着くかどうかで効果が決まっていたため。

> **評価は必ず全時間帯で行うこと。**ダイヤシフトは便を移すだけなので狙った時間帯は必ず改善するが、便を抜いた時間帯は悪化する。狙った時間帯だけで測ると「費用ゼロで改善」に見え、**全日で純悪化する施策（絵鞆循環線のシフトは7時台で4,932人減るが全日では3,694人・時間の悪化）を有益と誤認する。**
> **対象路線を便数で選んではいけない。**最多便の鉄北北口線は129停留所を通るが空白地域は0箇所で、増便しても1人も減らない。`rank_routes_by_blank_coverage()` で空白地域の停留所数により順位づけする。

費用は**運行キロ（便数×路線長）を代理値**とする。実額は運転士人件費・委託契約で決まり手元に無いが、バス事業のコストは慣例的に円/実車走行キロで表され補助金算定も走行キロ基準のため、単価を掛ければ金額になる。路線長は停留所間の直線距離の和で、実道路距離より短く出る（`shapes.txt` は未使用）。

8.2 他地域での実証（コード変更なし）

	室蘭（道南バス）	旭川（旭川電気軌道）	網走（網走バス）
停留所	2,182	765	583
便数（平日）	808	589	137
ネットワーク生成	4.1秒	1.0秒	0.3秒

空白人口の算出	✓	✓	人口データ未取得

旭川（拠点＝共栄バスセンター、60分以内、旭川電気軌道GTFS 2026年11月11日からのダイヤ）の平日9時台は平均 **177,702人（53.9%）**、内訳は静的空白88,570人＋時間空白89,132人。11,911メッシュ×216通りの診断が0.7秒。`python tools/key_numbers.py --region` で再計算できる。

9. 性能

処理	修正前	現在	手段
時間帯別到達圏（18回）	約30秒	**0.02秒**	グラフのキャッシュ化
人口計算（60停留所）	5.24秒	**0.011秒**	numpyベクトル化（結果一致を確認）
施策比較（4施策）	6.43秒	**1.50秒**	路線長のキャッシュ化

5分刻み化（4.4）後の実測：時間空白診断 216通り 約1秒、時間帯別到達圏 216回 0.5秒、施策比較 7施策×216通り 約8秒（Streamlit Cloud ではこの1.5〜2倍程度を見込む）。

探索本体は最初から0.00秒で、所要時間のほぼ全てはグラフ構築が占めていた。`iterrows` → `itertuples` の置換も併用。キャッシュは `bus_edges` / `walk_edges` 差し替え時に破棄する必要があり、`scenario()` と `set_day_type()` が面倒を見る。

10. テスト

合計1,558件（件数の一部は路線・施設ごとに生成されるため、データで変わる）。実行は各ファイルを直接 `python test_*.py`。

ファイル	件数	内容
test_all.py / _v2 / _v3	467 / 137 / 732	データ整合、到達圏、各シミュレーション、人口・施設連動、施設アクセスの歩行速度と制限時間
test_blank_area.py	56	空白判定の性質、500mメッシュ集約の保存則、5分刻みの平均と幅、人口と時刻表の地域のずれ
test_day_type.py	19	ダイヤ切替、キャッシュ破棄、単調性、ゼロ詰め service_id
test_scenario.py	33	scenario の適用・復帰・例外安全、`edges_after_*` と `simulate_*` の一致、減便の間引き方（方向別・N・入れ子・実際の率）、代替路線の帰り方向
test_prescription.py	31	候補順位づけ、増便・シフト（拠点基準の配置を含む）、費用中立性、全日評価の計算の一致、変化なしの判定
test_policy_report.py	58	文書の節と数値、判定の定義と幅、単位、推奨の選び方（同点・純悪化）、出典、数値照合、APIキーの扱い
test_session_isolation.py	25	訪問者間の分離（6.6の再現を含む）、アップロードの閉じ込め、不正なZIP

重要な不変条件

- **単調性**：便を増やせば到達範囲は広がる（土曜+平日 ⊇ 各々）。減便率を上げれば空白は増える（削る便が入れ子になるので構成上成り立つ）。時刻依存ダイクストラが壊れればここで検出できる
- **時間帯の値**：5分刻みの判定の平均で、メッシュ別の値を足すと一致し、最小と最大の間に入る。1時刻ずつの診断と行ごとに一致する
- **一致性**：`edges_after_X` + 探索 の結果が `simulate_X` と一致する
- **保存則**：500mメッシュ集約の前後で総人口・空白人口が保存される
- **費用中立**：ダイヤシフトで総便数・総運行キロが変わらない
- 静的空白は施策で変わらない（徒歩圏の問題なので）
- **分離**：ある訪問者のダイヤ切替・シナリオ適用中も、他の訪問者の結果は変わらない

11. 意図的に採らなかった選択

項目	判断と理由
デマンド交通	**機能ごと削除**。機能過多を避けるため。実装上も、デマンドを徒歩エッジとして追加していたため「徒歩の連続禁止」が適用され、徒歩→デマンド／デマンド→徒歩ができない歪みがあった。復活は git 履歴から可能
勾配（Tobler式・DEM）	**保留**。徒歩速度を一律半減（S=+0.20相当の最悪ケース）させても空白人口の増加は+9.1%にとどまる。全メッシュが急勾配という前提での上限値なので実際はより小さい。DEM取得の工数に見合うか未判断
250mメッシュ	**500mを採用**。100mメッシュは1kmの10等分のため250m（4等分）には整数で割り切れない。標準メッシュ体系で100mから作れるのは500m（5×5セル）
空間軸の処方を主軸に	**時間軸を主軸に**。「どこに路線を引くか」は2025年優秀賞「コミバスをつくろう！」と競合する。診断で時間空白を示しながら処方が空間軸では主張の軸がずれる
3Dグラフィックの生成AI利用	**不採用**。計算結果と一致しない図はEBPMツールの信頼性を損なう。柱の高さは計算値そのもの

12. 既知の限界

- `calendar_dates.txt` 未対応。祝日ダイヤは「日祝」のservice_idで近似
- 時刻表の有効期間：現在の道南バスGTFSは2026-12-25まで。冬ダイヤ等への改正があれば差し替え、`build_network.py` と `tools/key_numbers.py` を実行し直す
- 到着時刻は5分刻みで評価する（1分刻みとの差は休日7時台の平均で0.1ポイント）
- 家からバス停までは直線距離で歩くとみなす（道のり1.3倍で休日7:00着の空白が46.1%→53.8%）。乗り換えの余裕時間は0分
- 到達圏系の分析（到達圏・廃止・減便など）は、出発停留所名に複数の乗り場がある場合、先頭の乗り場から計算する（例：東室蘭駅西口10:00発は乗り場により322か356停留所）。空白診断の拠点はすべての乗り場を使う
- 「空白」は拠点に**行けるか**だけを見ており、帰りの便があるかは見ていない
- 徒歩エッジ生成が停留所の総当たり（室蘭で約271万組、2.8秒）。停留所数の2乗で増えるため、2万停留所規模では推計約3.4分かかる。GTFSごとに1回だけの前処理なので分析自体には影響しないが、大都市のGTFSをアップロードしたときの待ち時間になる
- 費用代理の路線長が直線距離のため実道路距離より短い。`shapes.txt` が未使用
- 人口は国勢調査の常住人口（夜間人口）を100mメッシュに按分した推計値。日中に人がどこにいるかは反映せず、1メッシュの値は実人数そのものではない
- 施設データは国土数値情報の医療機関・学校のみ。スーパー等の商業施設は含まない
- バス停削除時の徒歩再接続は、実測では到達数に影響しない（バス直通経路が支配的）。過度に重視しないこと
- アプリの3Dモードと処方モードが個別に診断を走らせるため計算が重複する（体感1秒程度）
- 政策文書のAI整形は、無効なキーでのエラー処理までは確認済みだが、有効なキーでAPIに受理されるかは未検証
- 無料枠の公開アプリは、しばらくアクセスが無いと休止し、最初の表示に時間がかかる

13. 操作リファレンス

```
# 起動
streamlit run app.py

# ネットワーク再生成（GTFSを差し替えたときのみ。全ダイヤ分を生成する）
python build_network.py

# 3Dマップ（Streamlit不要の単体HTML）を再生成 → docs/kuhaku-scope-muroran.html
python tools/build_map3d.py

# README・資料の数値を再計算（室蘭）／他地域の診断表
python tools/key_numbers.py
python tools/key_numbers.py --region <GTFSフォルダ> <人口CSV> <拠点の停留所名>

# テスト
python test_all.py / test_all_v2.py / test_all_v3.py
python test_blank_area.py / test_day_type.py / test_scenario.py / test_prescription.py
python test_policy_report.py / test_session_isolation.py
```

アプリのモードは10種：到達圏のみ／路線廃止／バス停削除／減便／時間帯別到達圏／施設アクセス／代替路線追加／集客圏分析／時間空白診断（3D）／処方（改善施策）。全モードで平日・土曜・日祝を切り替えられる。処方モードから政策文書の下書きを出力できる。
依存は pandas / numpy / streamlit / folium / streamlit-folium / pydeck / anthropic の7つで、動作確認した版に固定している（anthropicは政策文書のAI整形でのみ使い、無くても文書は出る）。

---

transit-accessibility ／ K-hskw ／ 本書の数値は道南バス GTFS-JP（2026年10月1日改正、ODPT基本ライセンス）と簡易100mメッシュ人口（令和2年国勢調査をもとに西澤明が作成、CC BY）による実測値。拠点・制限時間・徒歩条件を変えると値は変わる。